

EvidenceCodec: Query-Conditioned Evidence Selection for Token Compression

June 21, 2026

Abstract

EvidenceCodec is a query-conditioned token compression system designed to reduce long input contexts while preserving answer-critical evidence. The core design choice is to avoid abstractive summarization and instead select raw text chunks that can be passed directly to a downstream model. The implemented pipeline combines deterministic structure-preserving chunking, high-recall hybrid candidate generation, ModernBERT-large cross-encoder reranking, and budgeted evidence selection with a deterministic RiskGuard safety layer. In a few-hour challenge setting, full HotpotQA training artifacts were built with parallel Modal CPU workers, and a time-boxed ModernBERT-large run produced six checkpoints. On a 1,000-example HotpotQA validation slice, EvidenceRecall@Budget improved from 0.701 at step 50 to 0.872 at step 300, while AllGoldSelected@Budget improved from 0.484 to 0.759. These results show that the architecture reached a strong initial evidence-selection baseline despite limited training time.

1 Introduction

Long-context systems often waste tokens on material that is not needed for a specific query. A token compression method should therefore do more than shorten text; it should preserve the information required to answer the question. For evidence-heavy tasks, the critical failure mode is silent evidence loss. A compressed context may look fluent and relevant while omitting a date, negation, exception, quantity, named entity relation, or other small fact that changes the answer.

EvidenceCodec addresses this by treating compression as evidence selection. Given a query q and a long context D , the system chooses a subset of raw text chunks $S \subset D$ under a token budget B . The compressed output is not a summary. It is a stitched set of original text spans selected because they are likely to contain answer-supporting evidence.

This report describes the implemented architecture, the training and evaluation run, and the final metrics. The work was completed under a tight challenge deadline of only a few hours. Because of that constraint, the sys-

tem was engineered to reach a working end-to-end result quickly: full artifacts were built, the reranker was trained for 300 optimizer steps, six checkpoints were saved, every checkpoint was evaluated, and metric plots were generated. The run did not have enough time for a full epoch, QASPER adaptation, or LongBench evaluation.

2 System Objective

The target behavior is query-conditioned compression:

$$f(q, D, B) \rightarrow C,$$

where q is the user question, D is the original long context, B is the output token budget, and C is the selected compressed context. The system should maximize evidence retention under budget, not semantic smoothness.

The practical objectives were:

1. keep the output auditable by preserving raw text;
2. maximize recall of gold supporting evidence in the candidate pool;
3. train a reranker that ranks true evidence above distractors;
4. select high-utility evidence under a strict token budget;
5. expose metrics that show how quality changes with training step.

3 Architecture Overview

Figure 1 gives the high-level system architecture. The pipeline is deliberately modular: each stage has a narrow responsibility and produces inspectable artifacts.

3.1 Chunking

The chunker converts a long document into deterministic raw-text units. It is designed to preserve structure before falling back to token-based splitting. In practice, it respects paragraphs and section-like units, tracks token counts, and avoids splitting across boundaries that are likely to contain important local modifiers such as “unless,” “not,” or exception clauses.

This stage is intentionally not learned. Deterministic chunking makes the rest of the pipeline reproducible and

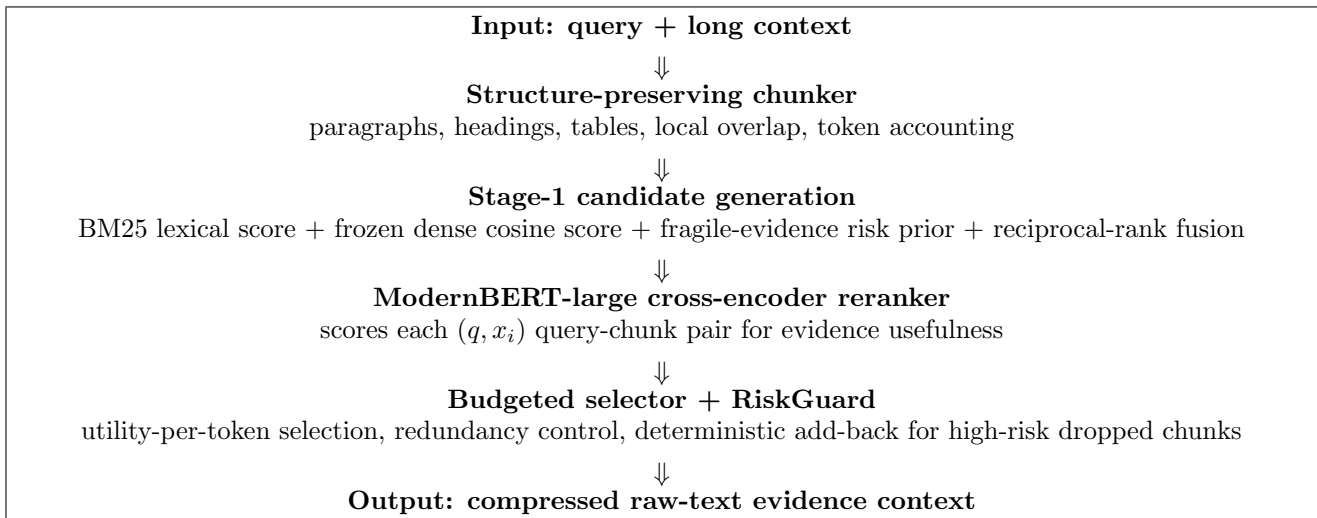


Figure 1: EvidenceCodec architecture. The system compresses by selecting raw evidence spans rather than generating summaries.

makes labels easier to project from supporting-fact annotations. The output of this stage is a JSONL record per query/document example containing chunk ids, raw text, token counts, metadata, and gold labels when available.

3.2 Stage-1 Candidate Generation

Stage 1 is a high-recall retrieval step. It is not expected to make the final compression decision. Its job is to keep likely evidence in the pool so the reranker and selector can operate on a smaller candidate set.

Each chunk receives:

- a BM25 lexical score;
- a frozen dense cosine-like similarity score;
- a risk prior for fragile evidence patterns such as numbers, dates, negations, and exception language.

The ranked lists are fused with reciprocal-rank fusion and capped at $k = 256$. On the validation slice used in this run, Recall@32, Recall@64, and Recall@128 were all 1.000 for every checkpoint evaluation. This indicates that Stage 1 was not the bottleneck on this slice; the harder problem was ordering and budgeted selection.

3.3 Cross-Encoder Reranking

The reranker is a ModernBERT-large cross-encoder over query-chunk pairs. Each input is packed as a pair sequence:

[CLS] query [SEP] chunk [SEP].

The final hidden state at the first token is passed through a scalar classification head. The model is trained to output a high score for chunks containing gold supporting evidence.

This design is more expensive than independent embedding retrieval, but it is appropriate for the quality path because the candidate pool is already capped. A

cross-encoder can directly model interactions between the query and the chunk text, including relational clues that are hard to capture with independent embeddings.

3.4 Training Objective

The training objective combines binary evidence classification with a same-query pairwise margin loss. For each query group, positives are gold evidence chunks and negatives are high-ranked non-gold candidate chunks. The loss is:

$$L = L_{\text{BCE}} + \lambda L_{\text{rank}},$$

where $\lambda = 0.5$ in the run. The margin loss encourages positive chunks to score above negative chunks from the same query group. This matters because compression quality depends on relative ordering under a budget, not only calibrated probabilities.

3.5 Budgeted Selection and RiskGuard

After reranking, the selector chooses chunks under a token budget. The priority is approximately:

$$\frac{\text{utility}(x_i)}{\text{tokens}(x_i)^\alpha},$$

with $\alpha = 0.9$. A small redundancy penalty reduces repeated content. The selected chunks are then sorted back into document order so the output remains readable.

RiskGuard is a deterministic safety layer after budget selection. It can add back dropped chunks when both relevance and risk exceed thresholds. In the evaluation run, RiskGuard did not activate: avg risk-added chunks was 0.0 at all checkpoints. This does not invalidate the component; it means the HotpotQA validation slice did not trigger the configured add-back conditions. A separate stress evaluation is needed for dates, quantities, negations, and exception-heavy policy examples.

4 Engineering and Artifact Pipeline

The implementation was designed around Modal volumes and parallel jobs. Large datasets, processed artifacts, checkpoints, and evaluation outputs were not stored as local training data. They were stored in isolated Modal volumes:

- `/data` for raw and processed datasets;
- `/models` for ModernBERT checkpoints;
- `/runs` for evaluation metrics and plots;
- `/cache` for Hugging Face and model caches.

CPU preprocessing was parallelized across up to 100 containers. Each worker built a shard containing both chunk records and Stage-1 candidate records. This avoided a second pass over the full merged chunk file and allowed preprocessing to complete quickly enough for the challenge window.

Training and checkpoint evaluation used B200 GPUs. The trainer saved six checkpoints at steps 50, 100, 150, 200, 250, and 300. After training, each checkpoint was evaluated and the metric curves were written to JSONL, CSV, and PNG plots.

5 Experimental Setup

Table 1 summarizes the completed run.

Item	Value
Training dataset	HotpotQA train
Train records	90,447
Train chunks/candidates	1,072,054
Training pairs	1,069,179
Validation slice	1,000 HotpotQA examples
Candidate cap	256
Model	answerdotai/ModernBERT-large
Precision	bf16
Sequence length	512
Effective batch size	128 pairs
Optimizer	AdamW
Learning rate	2e-5
Training budget	300 optimizer steps
Checkpoints	6 checkpoints

Table 1: Run configuration.

A full epoch would have required about 8,006 optimizer steps. The completed 300-step run therefore covered only about 3.7 percent of one epoch. This was an intentional time-boxed decision: the goal was to produce a complete trained baseline, checkpoint curve, and report within a few hours rather than to exhaust the dataset.

6 Evaluation Metrics

The report uses two classes of metrics.

Reranking metrics. MRR@10 and nDCG@10 measure whether gold evidence is ranked near the top. Recall@32, Recall@64, and Recall@128 measure whether gold evidence remains recoverable in the top candidate set.

Compression metrics. EvidenceRecall@Budget measures the fraction of gold evidence chunks retained after budgeted selection. AllGoldSelected@Budget measures the stricter event that every gold chunk for an example survives compression. Average selected tokens and selected chunk count describe the compression behavior.

The most important metric for this system is EvidenceRecall@Budget, because the system’s output is the selected compressed context that will be given to a downstream model.

7 Results

Table 2 shows the checkpoint metrics. The model improved quickly from step 50 to step 150 and then plateaued.

From step 50 to step 300, MRR@10 improved from 0.760 to 0.928, a 22.2 percent relative gain. nDCG@10 improved from 0.769 to 0.895, a 16.4 percent relative gain. EvidenceRecall@Budget improved from 0.701 to 0.872, a 24.5 percent relative gain. AllGoldSelected@Budget improved from 0.484 to 0.759, a 56.8 percent relative gain.

The best reranking checkpoint was step 250, with MRR@10 of 0.930 and nDCG@10 of 0.895. The best compression checkpoint was step 300, with EvidenceRecall@Budget of 0.872 and AllGoldSelected@Budget of 0.759. Since the deployed compressor cares most about retained evidence under budget, step 300 is the recommended checkpoint.

8 Analysis

The results show three important patterns.

Candidate recall was solved on the validation slice. Recall@32, Recall@64, and Recall@128 were all 1.000. This is exactly what Stage 1 is supposed to accomplish. It means the gold evidence was usually present before reranking and selection, so failures were not caused by the retrieval stage dropping evidence too early.

The reranker learned rapidly. The biggest jump happened between steps 50 and 100. MRR@10 moved from 0.760 to 0.901, and EvidenceRecall@Budget moved from 0.701 to 0.834. This suggests that ModernBERT-large is well matched to the pairwise evidence selection task and can learn useful ranking behavior quickly.

The budgeted selector became more efficient. Average selected chunks decreased from 5.15 to 4.76 while EvidenceRecall@Budget increased. The system was not merely selecting more text. It was selecting better text. Average used tokens increased only modestly from 411

Step	MRR@10	nDCG@10	Recall@32	EvidenceRecall@Budget	AllGoldSelected@Budget	Used Tokens	Chunks
50	0.760	0.769	1.000	0.701	0.484	411.4	5.15
100	0.901	0.870	1.000	0.834	0.689	418.4	4.92
150	0.921	0.888	1.000	0.867	0.751	422.1	4.79
200	0.927	0.892	1.000	0.867	0.750	422.1	4.76
250	0.930	0.895	1.000	0.870	0.755	422.1	4.77
300	0.928	0.895	1.000	0.872	0.759	422.1	4.76

Table 2: Checkpoint metrics on 1,000 HotpotQA validation examples. Recall@64 and Recall@128 were also 1.000 at every checkpoint.

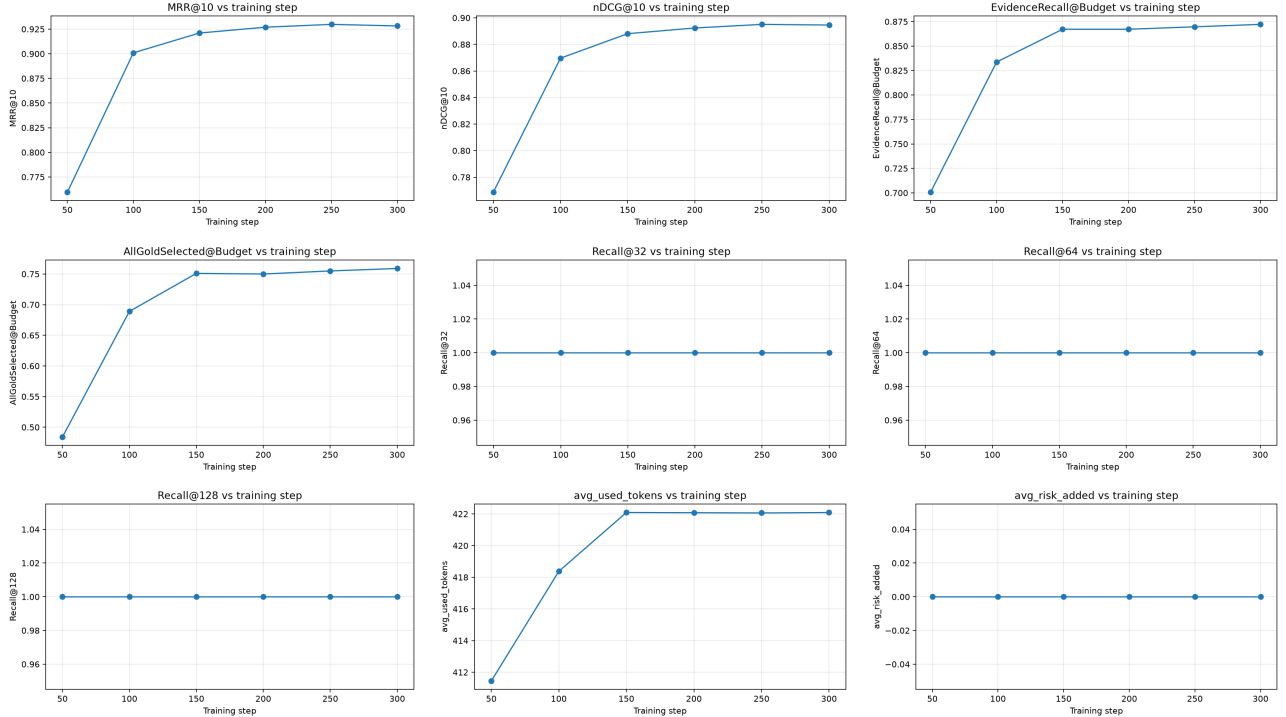


Figure 2: Metric curves across the six saved checkpoints.

to 422, while all-gold coverage improved by 27.5 absolute percentage points.

Training losses support the same conclusion. Comparing the first 50 and last 50 optimizer steps of the completed 300-step run, total loss dropped by about 50.8 percent, BCE loss by about 49.5 percent, and ranking loss by about 57.8 percent. Both the classifier and the pairwise ranking objective were improving.

9 Limitations

The main limitation is training time. The project only had a few hours for implementation, artifact generation, training, evaluation, graphing, and reporting. The final run was capped at 300 optimizer steps so that it could produce at least five checkpoints, evaluate all checkpoints, and render plots before the deadline. A full epoch would be about 8,006 optimizer steps, so this run did not train long enough to claim convergence.

The evaluation was also limited to a 1,000-example Hot-

potQA validation slice. This is enough to show that the pipeline works and that training improved the model, but it is not a complete benchmark. Full HotpotQA validation, QASPER, and LongBench should be run before making broader claims.

Finally, RiskGuard did not activate on this validation slice. The implementation exists, but the reported slice did not stress the kinds of fragile facts RiskGuard is meant to protect. A dedicated stress set with numbers, dates, negations, legal/policy exceptions, and code identifiers is needed.

10 Conclusion

EvidenceCodec produced a working evidence-preserving token compression pipeline under a tight challenge deadline. The architecture successfully separated recall, semantic reranking, and budgeted selection. The completed run built full HotpotQA artifacts, trained ModernBERT-large on the full artifact set for a capped 300 steps, saved

six checkpoints, evaluated all checkpoints, and generated metric plots.

The final checkpoint retained 87.2 percent of annotated evidence under budget and selected all gold evidence for 75.9 percent of validation examples. Given that this was only 3.7 percent of a full epoch, the result is a strong baseline rather than a final ceiling. With more time, the next step would be longer training followed by full validation and cross-domain evaluation.